Practical No. 14: * Write a program to handle data using React form

IX. Conclusion

In this practical, we learned how to handle form data in React using controlled components and state. React forms allow us to read user input, update the state, and display submitted data dynamically. This helps in building interactive and user-friendly web applications.

X. Exercise – Answers

1. Create a registration form that collects name, email, and password, and clears inputs after submission.

File- App.js

```
import Pract14_1 from "./Pract14_1";
import Pract14_2 from "./Pract14_2";

function App() {
 return <Pract14_1 />;
 //return <Pract14_2 />;

}
export default App;
```

File- Pract14_1.js

```
import React, { useState } from "react";

function Pract14_1() {

  // State variables for input fields
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  // State to store submitted data
  const [data, setData] = useState(null);

  // Function to handle form submit
  const handleSubmit = (e) => {
    e.preventDefault();

    // Store submitted data
    setData({ name, email, password });

    // Clear input fields
    setName("");
    setEmail("");
    setPassword("");
  };

  return (
    <div>
      <h2>Registration Form</h2>
      <form onSubmit={handleSubmit}>
        Name:
        <input
          type="text"
          value={name}
          onChange={(e)=>setName(e.target.value)}
        />
```

```
<br /><br />
 Email:
      <input
        type="email"
        value={email}
        onChange={(e)=>setEmail(e.target.value)}
      />
      <br /><br />

      Password:
      <input
        type="password"
        value={password}
        onChange={(e)=>setPassword(e.target.value)}
      />
      <br /><br />

      <button type="submit">Submit</button>
    </form>

    {/* Display Output */}
    {data && (
      <div>
        <h3>Submitted Data</h3>
        <p>Name: {data.name}</p>
        <p>Email: {data.email}</p>
        <p>Password: {data.password}</p>
      </div>
    )}

  </div>
 );
}

export default Pract14_1;
```

---

2. Demonstrate how to handle form data dynamically using an object to store multiple input fields.

File- Pract14_2.js

```
import React, { useState } from "react";
function Pract14_2() {

 // Object state to store multiple inputs
 const [formData, setFormData] = useState({
   name: "",
   email: ""
 });

 // State to store submitted data
 const [data, setData] = useState(null);

 // Update input values dynamically
 const handleChange = (e) => {
   setFormData({
     ...formData,
     [e.target.name]: e.target.value
   });
 };
```

```
// Handle form submit
 const handleSubmit = (e) => {
   e.preventDefault();

   // Store submitted data
   setData(formData);

   // Clear inputs
   setFormData({
     name: "",
     email: ""
   });
 };

 return (
   <div>

     <h2>Student Form</h2>

     <form onSubmit={handleSubmit}>

       Name:
       <input
         type="text"
         name="name"
         value={formData.name}
         onChange={handleChange}
       />
       <br /><br />

       Email:
       <input
         type="email"
         name="email"
         value={formData.email}
         onChange={handleChange}
       />
       <br /><br />

       <button type="submit">Submit</button>

     </form>

     {/* Display Output */}
     {data && (
       <div>
         <h3>Submitted Data</h3>
         <p>Name: {data.name}</p>
         <p>Email: {data.email}</p>
       </div>
     )}

   </div>
 );
}

export default Pract14_2;
```

XI. Practical Related Questions

1. How do you handle form data using React Hooks like useState and useEffect?

Form data in React is handled using the **useState hook** to store input values.
The **onChange event** updates the state whenever the user types in the input field.
The **useEffect hook** can be used to perform actions such as validation or API calls when the form data changes.

---

2. Demonstrate how to handle form data dynamically using an object to store multiple input fields.

Form data can be stored in a **single object using useState**. Each input field has a **name attribute**, and the handleChange() function updates the corresponding value using the spread operator.

Example:

```
const [formData, setFormData] = useState({
  name: "",
  email: ""
});
```

This approach makes form handling **simpler and scalable** for multiple input fields.